Algoritmos y Estructuras de Datos

Guía Práctica 7
Diseño y Elección de Estructuras
Segundo Cuatrimestre 2026



Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Diseño sobre listas

Ejercicio 1. Implemente el TAD ConjuntoAcotado⟨T⟩ usando la siguiente estructura.

```
TAD ConjAcotado⟨T⟩ {
    obs elems : conj⟨T⟩
    obs capacidad : ℤ
    proc conjVacío(in capacidad : ℤ) : ConjuntoAcotado⟨T⟩ {
        requiere { capacidad > 0 }
        asegura { res.elems = ⟨⟩ ∧ res.capacidad = capacidad }
    }
    proc igualdad(in C1 : ConjuntoAcotado⟨T⟩, in C2 : ConjuntoAcotado⟨T⟩) : bool {
        asegura {
            res = true ↔ |C1.elems| = |C2.elems| ∧L
                (∀e : T) (e ∈ C1.elems ↔ e ∈ C2.elems[i]) ∧
            C1.capacidad = C2.capacidad
        }
    }
    proc pertenece(in c : ConjAcotado⟨T⟩, in e : T) : bool {
        asegura { res = true ↔ e ∈ c.elems }
    }
    proc agregar(inout c : ConjAcotado⟨T⟩, in e : T) {
        requiere { c = C0 ∧ |c.elems| < c.capacidad }
        asegura { c.elems = C0.elems ∪ ⟨e⟩ ∧ c.capacidad = C0.capacidad }
    }
    proc sacar(inout c : Conjunto⟨T⟩, in e : T) {
        requiere { c = C0 }
        asegura { c.elems = C0.elems − ⟨e⟩ ∧ c.capacidad = C0.capacidad }
    }
    proc unir(inout c : Conjunto⟨T⟩, in c' : Conjunto⟨T⟩) {
        requiere { c = C0 ∧ |c.elems ∪ c'.elems| < c.capacidad }
        asegura { c.elems = C0.elems ∪ c'.elems ∧ c.capacidad = C0.capacidad }
    }
    proc restar(inout c : Conjunto⟨T⟩, in c' : Conjunto⟨T⟩) {
        requiere { c = C0 }
        asegura { c.elems = C0.elems − c'.elems ∧ c.capacidad = C0.capacidad }
    }
    proc intersecar(inout c : Conjunto⟨T⟩, in c' : Conjunto⟨T⟩) {
        requiere { c = C0 }
        asegura { c.elems = C0.elems ∩ c'.elems ∧ c.capacidad = C0.capacidad }
    }
    proc tamaño(in c : Conjunto⟨T⟩) : ℤ {
        asegura { res = |c.elems| }
    }
    proc capacidad(in c : Conjunto⟨T⟩) : ℤ {
        asegura { res = c.capacidad }
    }
}
```

Módulo ConjuntoArr<T> **implementa** ConjuntoAcotado$\langle T \rangle$ <

```
    var datos: Vector<T>
    var tamaño: int
```

>

a) Escriba el invariante de representación y la función de abstracción

b) Escriba los algoritmos para las operaciones

i) **conjVacío**
ii) **pertence**
iii) **agregar**
iv) **sacar**
v) **unir**
vi) **intersecar**

c) Calcule la complejidad de cada una de esas operaciones

d) ¿Qué cambios haría en su implementación si se quiere que la operación `agregar` sea lo más rápida posible? ¿Y si se quiere acelerar la operación `pertenece`? Indique los cambios en la estructura, el invariante de representación, la función de abstracción y los algoritmos.

Ejercicio 2. Implementar el TAD Conjunto$\langle T \rangle$ (definido en el apunte de TADs) usando la siguiente estructura

Módulo ConjuntoLista<T> **implementa** Conjunto$\langle T \rangle$ <

```
  var datos: ListaEnlazada<T>
  var tamano: int
```

>

a) Escriba el invariante de representación y la función de abstracción

b) Escriba los algoritmos para las operaciones `conjVacío` y `pertence`

i) **conjVacío**
ii) **pertence**
iii) **agregar**
iv) **agregarRapido**
v) **sacar**
vi) **unir**
vii) **intersecar**

c) Calcule la complejidad de cada una de estas operaciones

Diseño sobre árboles, *heaps* y *tries*

Ejercicio 3. Implementar los siguientes TADs sobre ABB. Calcule las complejidades de los procs en mejor y peor caso

a) Conjunto$\langle T \rangle$

b) Diccionario$\langle K, V \rangle$

c) ColaDePrioridad$\langle T \rangle$

Recalcule, si es necesario, las complejidades en peor caso de los algoritmos de los TADs considerando que se implementan sobre AVL en vez de ABB.

Ejercicio 4. Implemente el TAD ColaDePrioridad$\langle T \rangle$ utilizando heaps (implementados con arreglos).

Ejercicio 5. ¿Cómo haría para implementar una ColaDePrioridad ordenada por dos criterios? Por ejemplo, se quiere tener una cola de personas donde el criterio de ordenamiento es por edad y, en caso de empate, por apellido. Describa todos los cambios necesarios a la implementación del ejercicio anterior.

Ejercicio 6. Implemente el TAD Diccionario$\langle T \rangle$ utilizando *tries*.

Elección de estructuras

En esta sección nos concentraremos en la elección de estructuras y los algoritmos asociados. Cuando el enunciado diga "diseñe este módulo", salvo que indique lo contrario, sólo nos interesa:

1. La estructura
2. Su invariante de representación y su función de abstracción
3. Las operaciones con sus parámetros y su complejidad
4. Los algoritmos correspondientes

Ejercicio 7. Se desea diseñar un sistema para registrar las notas de los alumnos en una facultad. Al igual que en Exactas, los alumnos se identifican con un número de LU. A su vez, las materias tienen un nombre, y puede haber una cantidad no acotada de materias. En cada materia, las notas están entre 0 y 10, y se aprueban si la nota es mayor o igual a 7.

Materia, Alumno, Nota `ES` $\mathbb{Z}$
`TAD` Sistema {
  `obs` Notas : dicc$\langle Materia, \text{dicc}\langle Alumno, Nota\rangle\rangle$
  `proc nuevoSistema`() : $Sistema$
  `proc registrarMateria`(inout $s : Sistema$, in $m : Materia$)
  `proc registrarNota`(inout $s : Sistema$, in $m : Materia$, in $a : Alumno$, in $n : Nota$)
  `proc NotaDeAlumno`(in $s : Sistema$, in $a : Alumno, m : Materia$) : $Nota$
  `proc cantAlumnosConNota`(in $s : Sistema$, in $m : Materia, n : Nota$) : $\mathbb{Z}$
  `proc cantAlumnosAprobados`(in $s : Sistema$, in $m : Materia$) : $\mathbb{Z}$
}

Dados m la cantidad de materias y n la cantidad de alumnos, se desea diseñar un módulo con los siguientes requerimientos de complejidad temporal:

- nuevoSistema O(1)
- registrarMateria en O(log m)
- registrarNota en O(log n + log m)
- notaDeAlumno en O(log n + log m)
- cantAlumnosConNota y cantAlumnosAprobados en O(log m)

Ejercicio 8. El TAD *Matriz infinita de booleanos* tiene las siguientes operaciones:

- *Crear*, que crea una matriz donde todos los valores son falsos.
- *Asignar*, que toma una matriz, dos naturales (fila y columna) y un booleano, y asigna a este último en esa coordenada. (Como la matriz es infinita, no hay restricciones sobre la magnitud de fila y columna.)
- *Ver*, que dadas una matriz, una fila y una columna devuelve el valor de esa coordenada. (Idem.)
- *Complementar*, que invierte todos los valores de la matriz.

Ejemplo de uso del módulo:

```
MatrizInfinita M := Crear()
bool b1 := Ver(M, 0, 0)
Asignar(M, 1, 3, False)
Asignar(M, 100, 5000, True)
bool b2 := M.Ver(100, 5000)
Complementar(M)
bool b3 := Ver(M, 394, 788)
bool b4 := Ver(M, 100, 5000)
```

Tras lo que deberíamos tener

```
b1 = False
b2 = True
b3 = True
b4 = False
```

Elija la estructura y escriba los algoritmos de modo que las operaciones *Crear*, *Ver* y *Complementar* tomen $O(1)$ tiempo en peor caso.

Ejercicio 9. Se desea diseñar un sistema de estadísticas para la cantidad de personas que ingresan a un banco. Al final del día, un empleado del banco ingresa en el sistema el total de ingresantes para ese día. Se desea saber, en cualquier intervalo de días, la cantidad total de personas que ingresaron al banco. La siguiente es una especificación del problema.

```
TAD IngresosAlBanco {
    obs totales : seq⟨ℤ⟩
    proc nuevoIngresos() : IngresosAlBanco {
        asegura { totalDia = ⟨⟩ }
    }
    proc registrarNuevoDia(inout i : IngresosAlBanco, in cant : ℤ) {
        requiere { i = I_0 ∧ cant ≥ 0 }
        asegura { i.totales == I_0.totales ++ ⟨cant⟩ }
    }
    proc cantDias(in i : IngresosAlBanco) : ℤ {
        asegura { res == |i.totales| }
    }
    proc cantPersonas(in i : IngresosAlBanco, in desde : ℤ, in hasta : ℤ) : ℤ {
        requiere {
            0 ≤ desde ≤ hasta ≤ |i.totales|
        }
        asegura {
            res = Σ_{j=desde}^{hasta} i.totales[j]
        }
    }
}
```

1. Dar una estructura de representación que permita que la función *cantPersonas* tome $O(1)$.
2. Calcular cómo crece el tamaño de la estructura en función de la cantidad de días que pasaron.
3. Si el cálculo del punto anterior fue una función que no es $O(n)$, piense otra estructura que permita resolver el problema utilizando $O(n)$ memoria.
4. Agregue al diseño del punto anterior una operación *mediana* que devuelva el último (mayor) día d tal que cantPersonas$(i, 1, d)$ $\leq$ cantPersonas$(i, d + 1$,totDias$(i))$, restringiendo la operación a los casos donde dicho día existe.

Ejercicio 10. La Maderera San Blas vende, entre otras cosas, listones de madera. Los compra en aserraderos de la zona, los cepilla y acondiciona, y los vende por menor del largo que el cliente necesite.

Tienen un sistema un poco particular y ciertamente no muy eficiente: Cuando ingresa un pedido, buscan el listón más largo que tiene en el depósito, realizan el corte del tamaño que el cliente pidió, y devuelven el trozo que queda al depósito. Por otra parte, identifican a cada cliente con un código alfanumérico de 10 dígitos y cuentan con un fichero en el que registran todas las compras que hizo cada cliente (con la fecha de la compra y el tamaño del listón vendido).

Este sería el TAD simplificado del sistema:

```
Cliente ES string
TAD Maderera {
    proc comprarUnListon(inout m : Maderera, in tamaño : ℤ)
        //comprar en el aserradero un listón de un determinado tamaño

    proc venderUnListon(inout m : Maderera, in tamaño : ℤ, in cli : Cliente, in f : Fecha)
        // vender un listón de un determinado tamaño a un cliente particular en una fecha determinada

    proc ventasACliente(in m : Maderera, in cli : Cliente)
        // devolver el conjunto de todas las ventas que se le hicieron a un cliente
        // (para cada venta, se quiere saber la fecha y el tamaño del listón)
}
```

Se pide:

a) Escriba una estructura que permita realizar las operaciones indicadas con las siguientes complejidades:

   i) comprarUnListon en $O(\log(m))$

   ii) venderUnListon en $O(\log(m))$

   iii) ventasACliente en $O(1)$

   donde m es la cantidad de pedazos de listón que hay en el depósito

b) Escriba el algoritmo para la operación `venderUnListon`

Ejercicio 11. Se quiere implementar el TAD BIBLIOTECA que modela una biblioteca con su colección de libros. Se modela la biblioteca como una sola estantería de capacidad arbitraria, dentro de la cual cada libro ocupa una posición. La biblioteca cuenta con un registro de socios que pueden retirar y devolver libros en cualquier momento. Por restricciones del sistema que se utiliza, un socio no puede registrarse con un nombre de más de 50 caracteres.

Cuando la biblioteca adquiere un nuevo libro o cuando un libro es devuelto, éste es insertado en el primer espacio libre de la estantería. Es decir, si los lugares ocupados son 1, 2, 3, 4 y se presta el libro en la posición 2, al agregar un nuevo libro al catálogo éste será ubicado en la posición 2. Cuando el libro que estaba originalmente en la posición 2 sea devuelto, será ubicado en la primera posición libre, que será la 5.

El TAD tiene las siguientes operaciones, para las que se nos piden las complejidades temporales de peor caso indicadas, donde

- L es la cantidad de libros en la colección
- r es la cantidad de libros que el socio en cuestión tiene retirados
- k la cantidad de posiciones libres en la estantería
- `proc agregarLibroAlCatálogo`(inout $b : Biblioteca$, in $l : idLibro$)
  - **Requiere**: $\{l$ no pertenece a la colección de libros de $b\}$
  - **Descripción**: la biblioteca adquiere un nuevo libro, lo suma a su catálogo y lo pone en la estantería en el primer espacio disponible.
  - **Complejidad**: $O(log(k) + log(L))$
- `proc pedirLibro`(inout $b : Biblioteca$, in $l : idLibro$, in $s : Socio$)
  - **Requiere**: $\{s$ es socio de la biblioteca y el libro l no está entre los libros prestados$\}$
  - **Descripción**: el socio pasa a retirar un libro que se retira de la estantería y se acumula en sus libros prestados.
  - **Complejidad**: $O(log(r) + log(k) + log(L))$
- `proc devolverLibro`(inout $b : Biblioteca$, in $l : idLibro$, in $s : Socio$)
  - **Requiere**: $\{s$ es socio de la biblioteca y el libro l está entre sus libros prestados$\}$
  - **Descripción**: el socio pasa a devolver un libro que previamente había tomado prestado. Vuelve a la estantería en el primer espacio disponible.
  - **Complejidad**: $O(log(r) + log(k) + log(L))$

- `proc prestados`(in b : *Biblioteca*, in s : *Socio*) : Conjunto$\langle$*Libro*$\rangle$
  - **Requiere**: {s es socio de la biblioteca}
  - **Descripción**: este procedimiento retorna los libros que el socio tomó prestados de la biblioteca y aún no devolvió.
  - **Complejidad**: $O(1)$
- `proc ubicaciónDeLibro`(in b : *Biblioteca*, in l : *idLibro*) : *Posicion*
  - **Requiere**: {l pertenece a la colección de libros de b y no está prestado}
  - **Descripción**: obtiene la posición del libro en la estantería.
  - **Complejidad**: $O(log(L))$

Los datos se van a guardar en la siguiente estructura incompleta

```
  Socio es string; Libro, Posición es int

Módulo BibliotecaImpl implementa Biblioteca <

  /* Socios y los libros que tienen prestados */
  var socios: Diccionario<Socio, Conjunto<Libro>>

  /* Libros y su posición en la estantería */
  var catálogo: DiccionarioLog<Libro, Posición>

  /* Posiciones vacías en la estantería */
  var posicionesLibres: ColaDePrioridadLog<Posición>
>
```

a) Definir con qué estructura hay que implementar los diccionarios y conjuntos para cumplir las complejidades de las operaciones

b) Escribir los algoritmos mostrando cómo se cumplen las cotas de complejidad requeridas.

Ejercicio 12. Se quiere implementar el `TAD Agenda` que modela una agenda semanal donde se registran actividades. Cada actividad tiene un identificador, un horario de inicio y uno de finalización. No puede haber dos actividades con el mismo identificador. Para simplificar, las actividades sólo pueden comenzar y terminar en horarios en punto (por ejemplo, 21:00 hs) y terminan en un horario posterior a su inicio. Tampoco pueden empezar y terminar en días diferentes. Para contar la cantidad de actividades que transcurren en un determinado horario no se debe tener en cuenta aquellas que finalizan en ese momento. En cada actividad se pueden agregar tags que permiten agrupar las distintas actividades por temáticas. Los tags tienen cómo máximo 20 caracteres.

Dadas las siguientes operaciones y de acuerdo a las complejidades temporales de peor caso indicadas, donde

- d es la cantidad de días registrados hasta el momento
- a la cantidad total de actividades
- `proc RegistrarActividad`(inout ag : *Agenda*, in act : *IdActividad*, in $día$: *Día*, in $inicio$: *Hora*, in fin : *Hora*)
  1. **Requiere**: la hora de fin sea mayor que la de inicio y la actividad no está actualmente registrada.
  2. **Descripción**: se agrega la actividad a la agenda, marcando en el día indicado la hora de inicio y finalización.
  3. **Complejidad**: $O(\log(a) + \log(d))$
- `proc VerActividad`(in ag : *Agenda*, in act : *IdActividad*) : $struct < día : Día, inicio : Hora, fin : Hora >$
  1. **Requiere**: la actividad debe estar registrada en la agenda.
  2. **Descripción**: se devuelven el día y horario en que se realiza la actividad.
  3. **Complejidad**: $O(\log(a))$.
- `proc AgregarTag`(inout ag : *Agenda*, in act : *IdActividad*, in t : *Tag*)
  1. **Requiere**: la actividad está registrada en la agenda y aún no tiene registrado ese tag.
  2. **Descripción**: se agrega el tag a la actividad indicada.

3. **Complejidad**: $O(1)$.

- `proc HoraMásOcupada`(in $ag : Agenda$, in $d : Día$) : $Hora$)

  1. **Requiere**: el día indicado tiene al menos una actividad registrada.
  2. **Descripción**: se devuelve la hora que tiene más actividades registradas.
  3. **Complejidad**: $O(\log(d))$.

- `proc ActividadesPorTag`(in $ag : Agenda$, in $t : Tag$) : $Conjunto < IdActividad >$

  1. **Requiere**: true.
  2. **Descripción**: se devuelven las actividades que tienen registrado el tag t.
  3. **Complejidad**: $O(1)$.

1. Definir la estructura de representación del módulo `AgendaImpl`, que provea las operaciones solicitadas.
2. Escribir en castellano el invariante de representación.
3. Escribir los algoritmos de todas las operaciones, explicando cómo se cumplen las complejidades pedidas para cada una.

Ejercicio 13. Un instituto de cocina quiere organizar las inscripciones a su Escuela Culinaria de Invierno (ECI) donde ofrecen varias materias con capacidad limitada y nombres únicos de largo acotado. Los alumnos pueden intentar inscribirse a las materias y el sistema les informará si han quedado inscriptos o no (en caso que los inscriptos a la materia ya alcanzaran su capacidad). Si no pudieran inscribirse, igualmente quedan registrados en el sistema y serán agregados a la materia si algún inscripto se da de baja, por estricto orden de inscripción.

El sistema deberá implementar las siguientes operaciones en la complejidad pedida, donde a_m es la cantidad de alumnos inscriptos a la materia m. En todos los casos se puede asumir que la materia está registrada en el sistema. Materia es `string`. Alumno y Capacidad son `int`

- `proc abrirMateria`(inout $i : Inscripciones$, in $m : Materia$, in $c : Capacidad$)
  - **Descripción**: Abre la inscripción a una materia
  - **Complejidad**: $O(1)$
- `proc IntentarInscripcion`(inout $i : Inscripciones$, in $m : Materia$, in $a : Alumno$) : bool
  - **Descripción**: Intenta inscribir a un alumno a una materia a la que no está inscripto. Devuelve si el alumno quedó inscripto o no
  - **Complejidad**: $O(log(a_m))$
- `proc DarseDeBaja`(inout $i : Inscripciones$, in $m : Materia$, in $a : Alumno$)
  - **Descripción**: Saca al alumno de los inscriptos (donde ya debería estar) y agrega al siguiente de la lista de espera
  - **Complejidad**: $O(log(a_m))$
- `proc InscriptosMateria`(in $i : Inscripciones$, in $m : Materia$) : $Conjunto(Alumno)$
  - **Descripción**: Devuelve el conjunto de alumnos inscriptos en la materia
  - **Complejidad**: $O(1)$

a) Plantear la estructura de representación del módulo `ECIInscripcionesImpl`, que provea las operaciones mencionadas cumpliendo las complejidades requeridas.

b) Escribir el algoritmo de `DarseDeBaja`. Justifique **detalladamente** que se cumple la complejidad requerida.

Ejercicio 14. Como saben que somos muy buenos haciendo nuestro trabajo, nos piden ayuda desde un **Museo** para poder organizar las colecciones que poseen. En cada colección se registran las piezas que serán expuestas al público en algún momento. Una vez registradas, un curador analiza cada obra para poder determinar el año de su creación entre otras cosas. Los especialistas saben que las únicas obras realmente valiosas para sus colecciones datan aproximadamente desde el 4000 a.C. hasta el 2025 d.C. Finalmente, es de interés para el museo conocer cuáles son las obras que pertenecen a un año particular y cuál es la obra más antigua de cada una de sus colecciones para poder hacer difusión y atraer al público.

El sistema deberá implementar las siguientes operaciones en la complejidad pedida, donde c es la cantidad de colecciones registradas, p es la cantidad de piezas. *Colección*, *Pieza*, *Año* son *int*.

- `proc AbrirMuseo`() : $Museo$

- **Descripción**: Inicia el museo sin colecciones ni piezas registradas
- **Complejidad**: $O(1)$

- `proc PrepararColección`(inout $m : Museo$, in $c : Coleccion$, in $ps : Conjunto(Pieza)$)
  - **Descripción**: Se registra la colección con las piezas recibidas en ps sin las etiquetas de los años
  - **Requiere**: El conjunto contiene al menos una pieza
  - **Complejidad**: $O(log(c) + p.log(p))$
- `proc RegistrarPieza`(inout $m : Museo$, in $c : Coleccion, ,$ in $p : Pieza, ,$ in $a : Año$)
  - **Descripción**: Se registra el año de la pieza
  - **Requiere**: La pieza está registrada en la colección y el año está en el rango indicado
  - **Complejidad**: $O(log(c) + log(p))$
- `proc PiezasPorAño`(in $m : Museo$, in $a : Año$) : $Conjunto(Pieza)$
  - **Descripción**: Devuelve las piezas creadas en ese año
  - **Complejidad**: $O(1)$
- `proc KPiezasMásAntiguaPorColección`(in $m : Museo$, in $c : Coleccion$, in $k : int$) : $Conjunto\langle Pieza\rangle$
  - **Descripción**: Devuelve las k piezas más antiguas de la colección
  - **Requiere**: k toma valores entre 0 y la cantidad de piezas en la colección
  - **Complejidad**: $O(log(c) + k.log(p))$

Se pide:

a) Plantear la estructura de representación del módulo `Museo` que permita cumplir con las complejidades requeridas. **Justificar** con palabras de forma concisa las operaciones **PrepararColección**, **PiezasPorAño** y **KPiezasMásAntiguaPorColección**.

b) Escribir el algoritmo de la operación `RegistrarPieza`. Justifique **detalladamente** que se cumple la complejidad obtenida.